Στατιστική Μοντελοποίηση και Αναγνώριση Προτύπων 2023-2024 (ΤΗΛ311)

Εργασία 2

Φοιτητής 1: Αργύρης Γεώργιος 2020030138

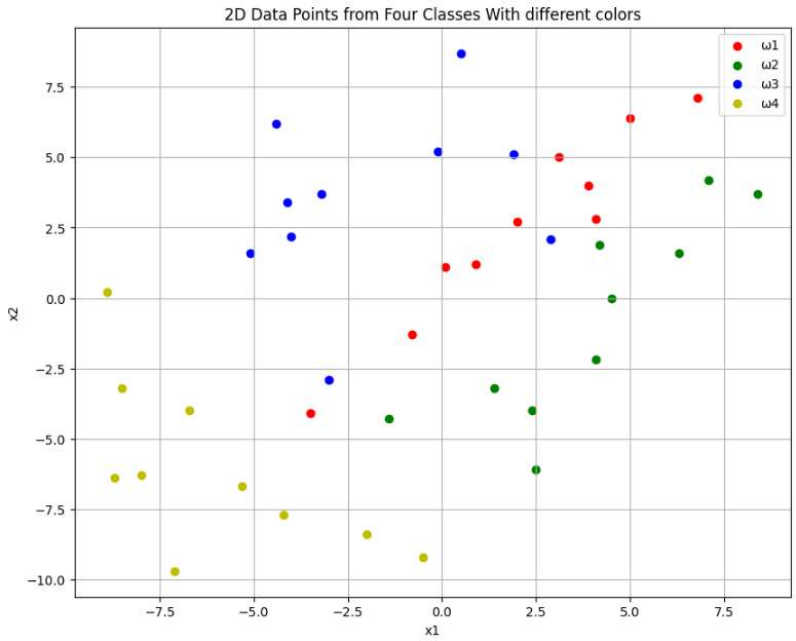
Φοιτητής 2: Αλεξίου Σταμάτιος 2020030158

04/07/2024

Θέμα 1: Αλγόριθμος Perceptron

Για το θέμα αυτό γράψαμε ένα κώδικα σε Python ο οποίος υλοποιεί τον αλγόριθμο Perceptron (στην batch μορφή του) για να κάνει ταξινόμηση ανάμεσα σε δύο κλάσεις. Πιο συγκεκριμένα :

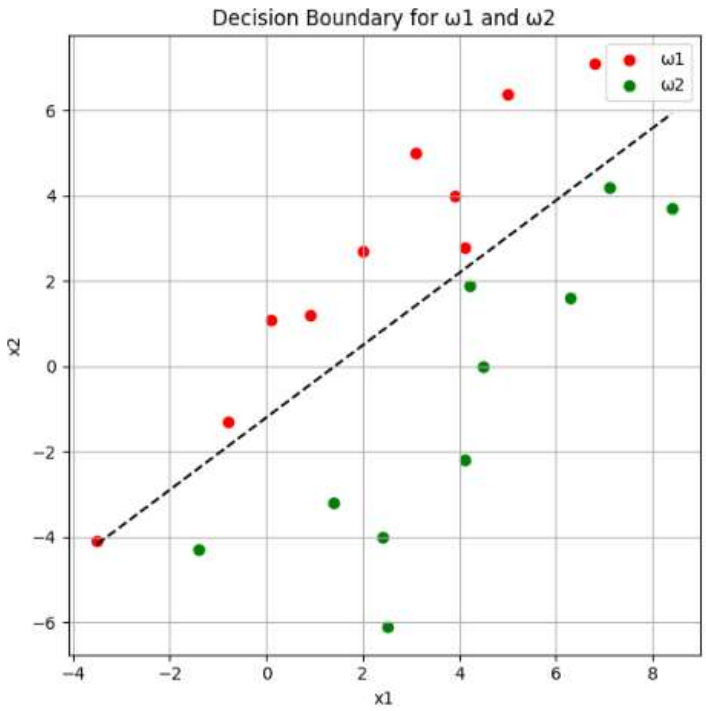
a) Διαβάσαμε και σχεδιάσαμε στο ίδιο διάγραμμα τα παραπάνω δείγματα, με διαφορετικό χρώμα για κάθε κλάση.



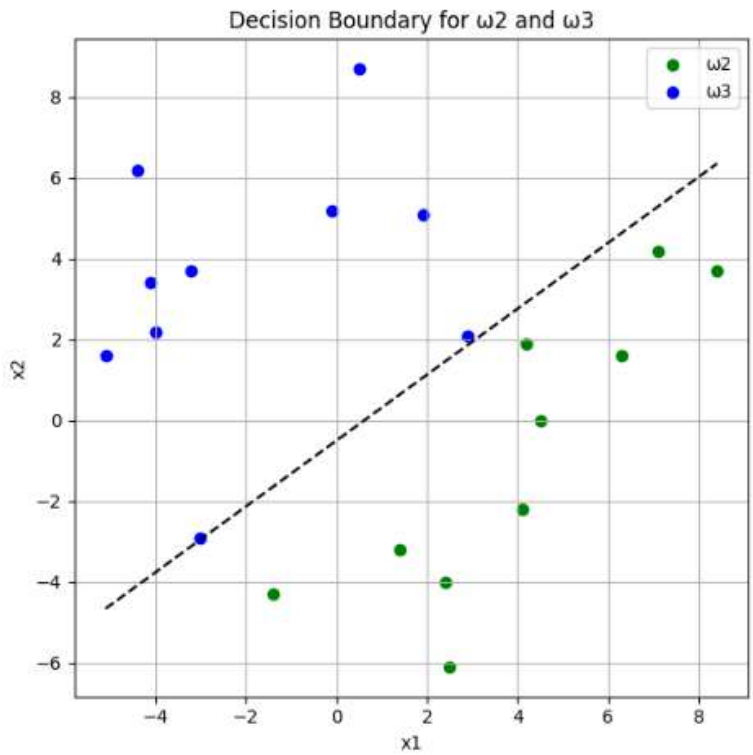
b, c) Ξεκινήσαμε με μηδενικές τιμές για το διάνυσμα των βαρών, και αφού δημιουργήσαμε τον αλγόριθμο, τον εφαρμόσαμε διαδοχικά για ταξινόμηση των δειγμάτων των κλάσεων $\omega 1$ και $\omega 2$, στη συνέχεια των κλάσεων $\omega 2$ και $\omega 3$ και τέλος των κλάσεων $\omega 3$ και $\omega 4$. Σε όλες τις περιπτώσεις σχεδιάσαμε το όριο απόφασης (decision boundary) και σημειώσαμε τον αριθμό των επαναλήψεων που απαιτούνται μέχρι να συγκλίνει ο αλγόριθμος για κάθε εφαρμογή.

Τα αποτελέσματα παρουσιάζονται στα παρακάτω στιγμιότυπα:

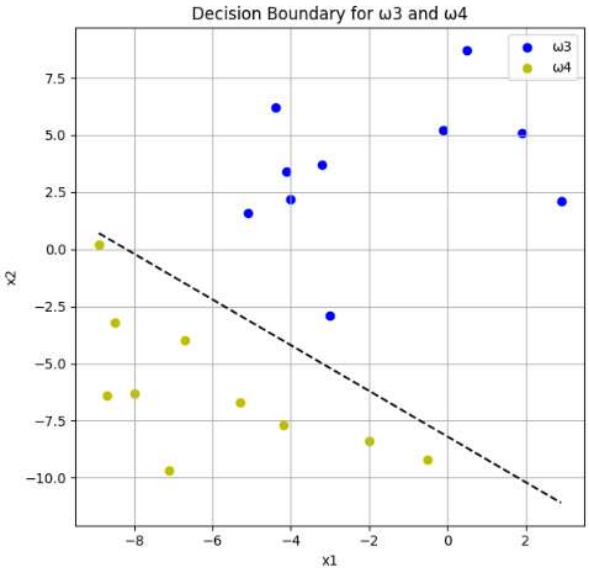
Εφαρμογή ω1 - ω2



Εφαρμογή ω2 - ω3



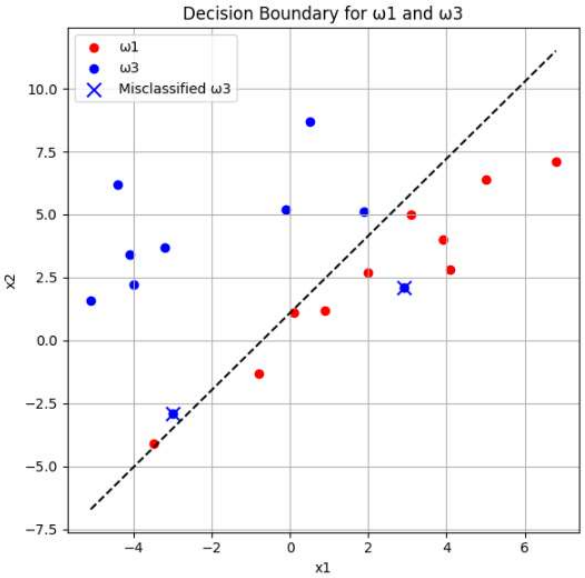
Εφαρμογή ω3 - ω4



Αριθμός απαιτούμενων επαναλήψεων προκειμένου να συγκλίνει ο αλγόριθμος για κάθε περίπτωση :

```
Number of iterations until convergence for ω1 and ω2: 29
Number of iterations until convergence for ω2 and ω3: 19
Number of iterations until convergence for ω3 and ω4: 59
```

Παρατήρηση 1: Αρχικά παρατηρούμε ότι δεν έχουμε misclassified data σε καμία από τις 3 περιπτώσεις που ελέγξαμε. Ο κώδικας που γράψαμε κάνει έλεγχο για misclassified data και τα σχεδιάζει ανάλογα . Παράδειγμα ταξινόμησης δειγμάτων που υπάρχει overlapping και misclassified data(ω1 – ω3) με learning rate = 0.5 και 10000 maximum iterations:



Παρατήρηση 2: Σχετικά με τον απαιτούμενο αριθμό επαναλήψεων που χρειάζεται κάθε εφαρμογή προκειμένου να συγκλίνει ο αλγόριθμος, βλέπουμε ότι υπάρχουν μικρές διαφορές μεταξύ τους, με την τελευταία εφαρμογή (ω3-ω4) να απαιτεί και τις περισσότερες επαναλήψεις. Αυτή η συμπεριφορά οφείλεται στο ποσοστό του overlapping μεταξύ δειγμάτων των δυο κλάσεων καθώς και γενικά στο πόσο καθαρό είναι το decision boundary κάθε φορά. Συγκεκριμένα:

- ω1 – ω2: Διακριτό decision boundary , και κατ' επέκταση λίγες επαναλήψεις.

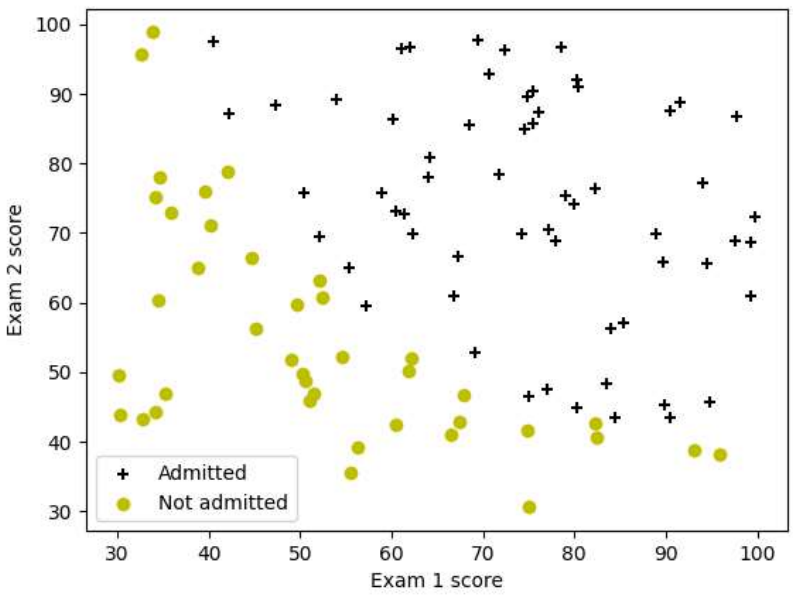
- ω2 – ω3: Καθαρός διαχωρισμός κλάσεων και πολύ λίγες επαναλήψεις του αλγορίθμου.

- ω3 – ω4: Λιγότερο διακριτός διαχωρισμός δειγμάτων και συνεπώς περισσότερες επαναλήψεις μέχρι να επιτευχθεί η σύγκληση.

Θέμα 2: Λογιστική Παλινδρόμηση: Αναλυτική εύρεση κλίσης (Gradient)

Σκοπός της άσκησης ήταν να χρησιμοποιήσουμε την λογιστική παλινδρόμηση για να προβλέψουμε αν ένας φοιτητής θα γίνει δεκτός σε ένα πανεπιστήμιο με βάση τους βαθμούς του σε δύο εξετάσεις, χρησιμοποιώντας ως εκμάθησης της λογιστικής παλινδρόμησης δεδομένα από παλαιότερες αιτήσεις φοιτητών που βρίσκονται στο αρχείο exam_scores_data1.txt στη μορφή Exam1Score, Exam2Score, [0: απόρριψη, 1: αποδοχή].

Αρχικά, διαβάσαμε το αρχείο και σχεδιάσαμε τα δείγματά μας.



Στην συνέχεια, συμπληρώσαμε τον κώδικα της συνάρτησης sigmoid ώστε να υλοποιεί την σιγμοειδή συνάρτηση $f(z)$ της άσκησης η οποία γνωρίζουμε ότι ισούται με: $f(z) = \frac{1}{1+2e^{-z}}$.

```
def sigmoid(z):
    sigmoid_function = 1 / (1 + 2*np.exp(-z))
    return sigmoid_function
```

Έπειτα, συμπληρώσαμε τις συναρτήσεις costFunction και gradient έτσι ώστε να επιστρέφει το κόστος $J(\boldsymbol{\theta})$ (J) και την κλίση $\nabla J(\boldsymbol{\theta})$ (grad).

Η συνάρτηση κόστους/σφάλματος (loss function), ονομάζεται cross-entropy, και ορίζεται ως εξής:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}(-y^{(i)}\, ln(\hat{y}^{(i)}) - (1-y^{(i)})ln(1-\hat{y}^{(i)}))$$

όπου $\hat{y}^{(i)} = h_\theta\left(x^{(i)}\right) = f(\theta^T x^{(i)}) = \frac{1}{1+2e^{-\theta^T x}}$

```
# Calculate the cost function
def costFunction(theta, X, y):
    m, n = X.shape # same as y.size
    h = sigmoid(np.dot(X, theta)) # Calculates f(θT*x)
    J = (1/m) * (-y @ np.log(h) - (1 - y) @ np.log(1 - h))
    return J
```

Πριν υλοποιήσουμε την κώδικα της κλίσης θα αποδείξουμε ότι η κλίση του σφάλματος για το j στοιχείο ισούται με:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})\, x_j{}^{(i)}$$

$$\text{Έχουμε: } J(\theta) = \frac{1}{m}\sum_{i=1}^{m}(-y^{(i)}\, ln(\hat{y}^{(i)}) - (1-y^{(i)})ln(1-\hat{y}^{(i)}))$$

$$\text{Οπότε: } \frac{\partial J(\theta)}{\partial \theta_j} = \frac{\partial J(\theta)}{\partial \theta_j}\left(\frac{1}{m}\sum_{i=1}^{m}[(-y^{(i)}\, ln(\hat{y}^{(i)}) - (1-y^{(i)})ln(1-\hat{y}^{(i)})]\right)$$

$$= \frac{1}{m}\sum_{i=1}^{m}\frac{\partial J(\theta)}{\partial \theta_j}[(-y^{(i)}\, ln(\hat{y}^{(i)}) - (1-y^{(i)})ln(1-\hat{y}^{(i)})]$$

Αρχικά, υπολογίζουμε την παράγωγο της συνάρτησης κόστους ως προς $\hat{y}^{(i)}$.

$$\frac{\partial J(\theta)}{\partial \hat{y}^{(i)}}[-y^{(i)}\, ln(\hat{y}^{(i)}) - (1-y^{(i)})ln(1-\hat{y}^{(i)}) = -\frac{y^{(i)}}{\hat{y}^{(i)}} + \frac{1-y^{(i)}}{1-\hat{y}^{(i)}}$$

Γνωρίζουμε ότι: $\hat{y}^{(i)} = \frac{1}{1+2e^{-\theta^T x^{(i)}}}$

Εφαρμόζοντας chain-rule και θέτοντας ως z = $\theta^T x^{(i)}$ έχουμε:

$$\frac{\partial \hat{y}^{(i)}}{\partial \theta_j} = \frac{\partial \hat{y}^{(i)}}{\partial z}\frac{\partial z}{\partial \theta_j}$$

$$\frac{\partial \hat{y}^{(i)}}{\partial z} = \frac{\partial}{\partial z}\left(\frac{1}{1+2e^{-z}}\right) = \frac{2e^{-z}}{(1+2e^{-z})^2} = \hat{y}^{(i)}(1-\hat{y}^{(i)})$$

$$\frac{\partial z}{\partial \theta_j} = x_j{}^{(i)} \ (\textit{Αφού}, z = \theta_1 x_1{}^{(i)} + \ldots + \theta_j x_j{}^{(i)} + \ldots + \theta_n x_n{}^{(i)})$$

Συνεπώς, $\frac{\partial \hat{y}^{(i)}}{\partial \theta_j} = \frac{\partial \hat{y}^{(i)}}{\partial z}\frac{\partial z}{\partial \theta_j} = \hat{y}^{(i)}\left(1-\hat{y}^{(i)}\right) x_j{}^{(i)}$

Συνολικά με χρήση chain-rule έχουμε :

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m}\sum_{i=1}^{m}\frac{\partial J(\theta)}{\partial \hat{y}^{(i)}}\frac{\partial \hat{y}^{(i)}}{\partial \theta_j} =$$

$$= \frac{1}{m}\sum_{i=1}^{m}\left(-\frac{y^{(i)}}{\hat{y}^{(i)}} + \frac{1-y^{(i)}}{1-\hat{y}^{(i)}}\right)\left(\hat{y}^{(i)}(1-\hat{y}^{(i)})\, x_j{}^{(i)}\right) = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right) x_j{}^{(i)} =$$

$$= \frac{1}{m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right) x_j{}^{(i)} = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right) x_j{}^{(i)}$$

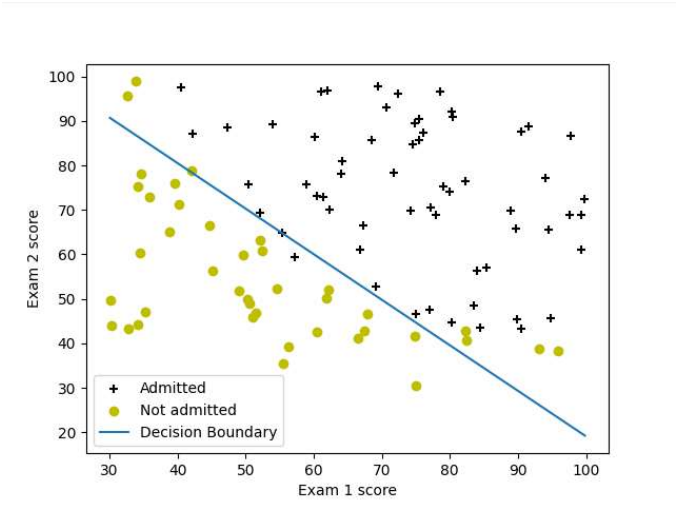
Παρακάτω φαίνεται ο κώδικας της συνάρτησης gradient:

```
# Calculate the gradient of the cost function
def gradient(theta, X, y):
    m = y.size # same as m.n X.shape
    h = sigmoid(np.dot(X, theta)) # Calculates f(θT*x)
    grad = (1/m) * np.dot(X, (h - y))
    return grad
```

Τέλος, συμπληρώσαμε τον κώδικα στην συνάρτηση predict για να προβλέψουμε αν ο φοιτητής θα γίνει δεκτός με βάση διάφορες τιμές βαθμών στις δύο εξετάσεις. Θέσαμε ως threshold value το 0.5 το οποίο αποτελεί την τυπική τιμή.

```
def predict(theta, X):
    probability = sigmoid(np.dot(X, theta))
    class_pred = probability >= 0.5
    return class_pred
```

Η βελτιστοποίηση των παραμέτρων έγινε με κώδικα που υπήρχε έτοιμος στην άσκηση. Οπότε, απλά τρέξαμε τον κώδικα και βρήκαμε το σύνορο απόφασης με την συνάρτηση plotDecisionBoundary. Επίσης, εκτυπώσαμε το κόστος και την κλίση της συνάρτησης κόστους πριν και μετά της βελτιστοποίησης μέσω minimization.




```
Cost at initial theta (zeros): 0.8213534164441316
Gradient at initial theta (zeros): [ -0.26666667 -22.94992893 -22.29984189]
Cost at theta found by minimize: 0.20349770159239444
theta: [-24.46817402   0.20623133   0.2014718 ]
```

Παρατηρούμε ότι το κόστος μετά την βελτιστοποίηση έχει μειωθεί αρκετά και η κλίση της συνάρτησης έχει αλλάξει.

Τέλος, ελέγξαμε το αποτέλεσμα για έναν φοιτητή που έγραψε στο πρώτο μάθημα 45 και στο δεύτερο 85, ενώ εκτυπώσαμε επίσης και το ποσοστό ακρίβειας της εκπαίδευσής μας.

```
For a student with scores 45 and 85, we predict an admission probability of 0.7762927969028834
Train Accuracy: 89.0
```

Συνεπώς, ο φοιτητής έχει περίπου 77.63% πιθανότητα να γίνει αποδεκτός με τους συγκεκριμένους βαθμούς, ενώ η ακρίβεια μας είναι ίση με 89% που είναι ένα πάρα πολύ ικανοποιητικό ποσοστό.

Θέμα 3: Εκτίμηση Παραμέτρων με Maximum Likelihood

Σε αυτή την άσκηση, έχοντας σαν δεδομένα 2 κλάσεις (ω1 , ω2), εκ των οποίων η κάθε μια έχει 3 χαρακτηριστικά (x1 , x2 , x3), γράψαμε ένα κώδικα σε Python ο οποίος υλοποιεί-απαντάει στα παρακάτω ερωτήματα:

a) Υπολογισμός μέγιστης πιθανοφάνειας μ $και$ σ^2 . Εφαρμογή προγράμματος ξεχωριστά σε κάθε ένα από τα 3 χαρακτηριστικά της κλάσης ω1. Αποτελέσματα:

```
------------------------ a ------------------------
Feature x_1:
  Mean: -0.07089999999999998
  Variance: 0.9061772900000001

Feature x_2:
  Mean: -0.6829999999999999
  Variance: 4.174821

Feature x_3:
  Mean: -0.9109999999999999
  Variance: 4.541949000000001
```

b) Υπολογισμός παραμέτρων κανονικής κατανομής με την μέθοδο μέγιστη πιθανοφάνειας , αν θεωρήσουμε ότι ανά δυο τα χαρακτηριστικά της ω1 ακολουθούν 2D κανονική κατανομή $\boldsymbol{p}(\boldsymbol{x})\sim\boldsymbol{\mathcal{N}}(\boldsymbol{\mu},\ \boldsymbol{\Sigma})$, $\boldsymbol{x} \in \mathbb{R}^2$. Εφαρμογή του προγράμματος και στα 3 ζευγάρια χαρακτηριστικών που υπάρχουν. Αποτελέσματα:

```
------------------------ b ------------------------
Pair 1 (x_1, x_2):
  Mean vector: [-0.0709 -0.683 ]
  Covariance matrix:
[[0.90617729 0.5293443 ]
 [0.5293443  4.174821  ]]

Pair 2 (x_1, x_3):
  Mean vector: [-0.0709 -0.911 ]
  Covariance matrix:
[[0.90617729 0.3940801 ]
 [0.3940801  4.541949  ]]

Pair 3 (x_2, x_3):
  Mean vector: [-0.683 -0.911]
  Covariance matrix:
[[4.174821 0.616957]
 [0.616957 4.541949]]
```

c) Υπολογισμός παραμέτρων κανονικής κατανομής με την μέθοδο της μεγίστης πιθανοφάνειας, θεωρώντας ότι το συνολικό διάνυσμα χαρακτηριστικών της κλάσης $\boldsymbol{\omega 1}$ ακολουθεί 3D κανονική κατανομή $\boldsymbol{p}(\boldsymbol{x})\sim\boldsymbol{\mathcal{N}}(\boldsymbol{\mu},\boldsymbol{\Sigma})$, $\boldsymbol{x} \in \mathbb{R}^3$.
Αποτελέσματα:

```
------------------------- c -------------------------
Mean vector : [-0.0709 -0.683  -0.911 ]
Covariance matrix :
[[0.90617729 0.5293443  0.3940801 ]
 [0.5293443  4.174821   0.616957  ]
 [0.3940801  0.616957   4.541949  ]]
```

d) Εκτίμηση με τη μέθοδο μέγιστης πιθανοφάνειας το μέσο $\boldsymbol{\mu}$ ̂και τα διαγώνια στοιχεία του $\boldsymbol{\Sigma}$ ̂, αν υποθέσουμε ότι το τρισδιάστατο μοντέλο σας είναι διαχωρίσιμο, έτσι ώστε $\boldsymbol{\Sigma}$ = $diag(\sigma 1^2, \sigma 3^2, \sigma 3^2)$.
Αποτελέσματα:

```
------------------------- d -------------------------
Mean vector: [-0.1126   0.4299   0.00372]
Diagonal covariance matrix:
[[0.05392584 0.         0.        ]
 [0.         0.04597009 0.        ]
 [0.         0.         0.00726551]]
```

Σημείωση: Στα παραπάνω ερωτήματα λόγω του ότι γίνεται εκτίμηση-υπολογισμός με την μέθοδο μέγιστης πιθανοφάνειας , η διασπορά υπολογίζεται με bias.

e) Για την σύγκριση αποτελεσμάτων έγινε δοκιμή υπολογισμού παραμέτρων χωρίς bias για τα ερωτήματα b),c) και d) .

Αποτελέσματα χωρίς bias:

b) Without Bias

```
---------------------- b ----------------------
Pair 1 (x_1, x_2):
  Mean vector: [-0.0709 -0.683 ]
  Covariance matrix:
[[1.00686366 0.58816033]
 [0.58816033 4.63869   ]]

Pair 2 (x_1, x_3):
  Mean vector: [-0.0709 -0.911 ]
  Covariance matrix:
[[1.00686366 0.43786678]
 [0.43786678 5.04661   ]]

Pair 3 (x_2, x_3):
  Mean vector: [-0.683 -0.911]
  Covariance matrix:
[[4.63869    0.68550778]
 [0.68550778 5.04661   ]]
```

c) Without Bias

```
---------------------- c ----------------------
Mean vector : [-0.0709 -0.683  -0.911 ]
Covariance matrix :
[[1.00686366 0.58816033 0.43786678]
 [0.58816033 4.63869    0.68550778]
 [0.43786678 0.68550778 5.04661   ]]
```

d) Without Bias

```
------------------------ d ------------------------
Mean vector: [-0.1126   0.4299   0.00372]
Diagonal covariance matrix:
[[0.0599176  0.         0.        ]
 [0.         0.05107788 0.        ]
 [0.         0.         0.00807278]]
```

<u>Παρατηρήσεις – Σύγκριση</u>

Βλέπουμε ότι όταν η εκτίμηση γίνεται με bias έχουμε ταύτιση τόσο του mean (μ) όσο και του variance (σ^2). Συγκεκριμένα, οι τιμές των μ και σ^2 παραμένουν ίδιες στο ερώτημα b) με αυτές που είχαν υπολογιστεί στο 1° ερώτημα (π.χ **Variance** of feature **1** (x1)(**from a)** = $\mathbf{S_{11}}$ of Pair 1 (x1,x2) (**from b)** = **0.90617729** και **Variance** of feature **2** (x2) (**from a)** = $\mathbf{S_{22}}$ of Pair 1 (x1,x2) (**from b)** = **4.174821**).

Αντίθετα, όταν η εκτίμηση γίνεται χωρίς bias η διασπορά αλλάζει και συγκεκριμένα γίνεται ελάχιστα μεγαλύτερη (π.χ **Variance** of feature 1 (x1)(**from a)** = **0.90617729** != $\mathbf{S_{11}}$ of Pair 1 (x1, x2) (from **b)** = **1.00686366** και **Variance** of feature 2 (x2) (from **a)** = **4.174821** != $\mathbf{S_{22}}$ of Pair 1 (x1,x2) (**from b)** = **4.63869**).

Οι **ίδιες αλλαγές στις τιμές** ισχύουν και για τά ερωτήματα **c)**,**d)**.Το **mean** παραμένει το **ίδιο** ακόμα και αν ο υπολογισμός γίνει **χωρίς bias**.

Θέμα 4: Ομαδοποίηση (Clustering) με K-means και GMM

Σε αυτή την άσκηση υλοποιήσαμε τον K-means clustering αλγόριθμο και τον χρησιμοποιήσαμε για να συμπιέσουμε μια εικόνα.

Ξεκινήσαμε συμπληρώνοντας τον κώδικα που λείπει στην συνάρτηση initialize_centroids όπου επιλέγουμε τυχαία K σημεία από το X (dataset).

```
# Randomly select initial centroids from the dataset.
def initialize_centroids(X, K):
    centroids = X[np.random.choice(X.shape[0], K, replace=False)]
    return centroids
```

Χρησιμοποιήσαμε την συνάρτηση choice με ορίσματα το X.shape[0] ως τον αριθμό των δειγμάτων στο dataset, το K για να δείξουμε πόσα δείγματα θέλουμε και το replace = False, ώστε να μην επιλεχθεί το ίδιο δείγμα δύο φορές.

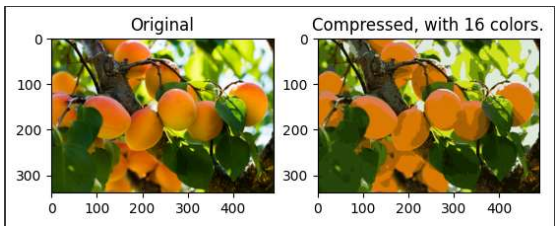
Συνεχίσαμε συμπληρώνοντας τον κώδικα που λείπει στην συνάρτηση findClosestCentroids η οποία βρίσκει τα $c^{(i)} := j$ που ελαχιστοποιούν την απόσταση $||x^{(i)} - \mu_j||^2$, όπου $c^{(i)}$ είναι ο δείκτης του κοντινότερου κέντρου στο $x^{(i)}$, και μ_j είναι το διάνυσμα τιμών (συντεταγμένων) του κέντρου j. Το $c^{(i)}$ αντιστοιχεί στο idx(i) του παραπάνω κώδικα.

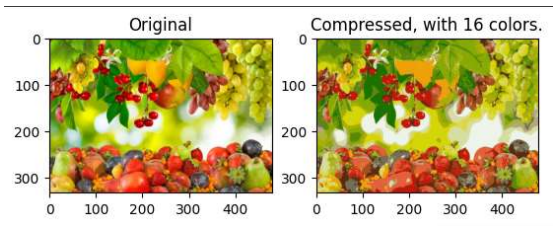
```
# Finds the closest centroid for each sample
def find_closest_centroids(X, centroids):
    m = X.shape[0]
    idx = np.zeros(m, dtype=int)
    for i in range(m):
        distances = np.linalg.norm(X[i] - centroids, axis=1)**2
        idx[i] = np.argmin(distances)
    return idx
```

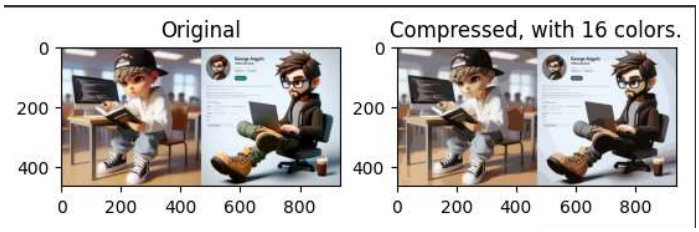
Έπειτα, συμπληρώσαμε την συνάρτηση computeCentroids η οποία υπολογίζει τα κεντροειδή των κλάσεων μ_K.

```
# Compute the mean of samples assigned to each centroid
def compute_centroids(X, idx, K):
    centroids = np.zeros((K, X.shape[1]))
    for k in range(K):
        points = X[idx == k]
        if points.shape[0] > 0:
            centroids[k] = np.mean(points, axis=0)
    return centroids
```

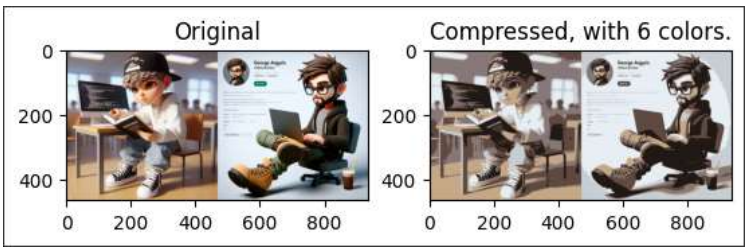
Τέλος, εφαρμόσαμε κανονικοποίηση στις τιμές της εικόνας στο εύρος 0 – 1 και αλλάξαμε τον κώδικα έτσι ώστε να τρέχει και σε εικόνες με 4 κανάλια (RGBA), καθώς το default ήταν 3 (RGB).

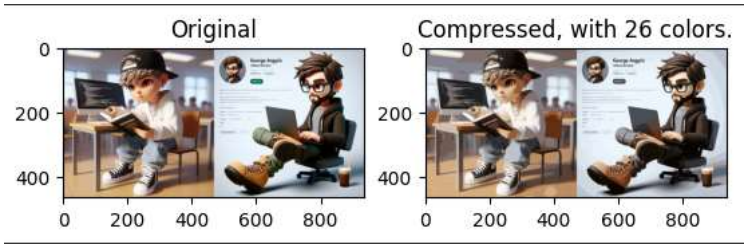






Ξανατρέξαμε την τελευταία φωτογραφία για K = 6 και K = 26





Ο αριθμός K καθορίζει πόσα διαφορετικά χρώματα (ή κλάσεις) θα χρησιμοποιηθούν για την αναπαράσταση της εικόνας. Ουσιαστικά, το K-means clustering συγκεντρώνει τα χρώματα της εικόνας σε K κεντροειδή χρώματα, και κάθε pixel της εικόνας αντικαθίσταται με το κοντινότερο κεντροειδές χρώμα.

Συνεπώς, όταν αυξάνετε το K, η ποιότητα της συμπιεσμένης εικόνας συνήθως βελτιώνεται. Αυτό συμβαίνει γιατί χρησιμοποιούνται περισσότερα χρώματα για την αναπαράσταση της εικόνας. Αντιθέτως, όταν μειώνετε το K (π.χ. σε 2, 4 ή 6), η ποιότητα της συμπιεσμένης εικόνας συνήθως μειώνεται. Αυτό συμβαίνει γιατί η εικόνα αναπαρίσταται με λιγότερα χρώματα, γεγονός που μπορεί να οδηγήσει σε σημαντική απώλεια λεπτομερειών και χρωματικών διαφορών.

Επίσης, ο όγκος των δεδομένων συνήθως μειώνεται με τη μείωση του K, καθώς επιτυγχάνεται μεγαλύτερη συμπίεση. Αυτό συμβαίνει γιατί τα κεντροειδή χρώματα απαιτούν λιγότερη μνήμη και η αναπαράσταση της εικόνας με λιγότερα χρώματα απαιτεί λιγότερα δεδομένα για την αποθήκευση των συντελεστών των χρωμάτων. Αντιθέτως, η χρήση μεγαλύτερου K συνήθως αυξάνει τον όγκο των δεδομένων, καθώς επιτυγχάνεται μικρότερη συμπίεση. Αυτό συμβαίνει γιατί τα κεντροειδή χρώματα χρειάζονται περισσότερη μνήμη και η αναπαράσταση της εικόνας με περισσότερα χρώματα απαιτεί περισσότερα δεδομένα για την αποθήκευση των συντελεστών των χρωμάτων.

Θέμα 5: Νευρωνικά δίκτυα

Στο Θέμα 5 επιλέξαμε να υλοποιήσουμε το Θέμα 5β.

Θέμα 5β: Convolutional Neural Networks for Image Recognition

Σε αυτή την άσκηση δοκιμάσαμε διάφορους ταξινομητές νευρωνικών δικτύων από την βιβλιοθήκη tensorflow/keras χρησιμοποιώντας το σύνολο δεδομένων Fashion-MNIST. Οι εικόνες του Fashion-MNIST χωρίζονται σε 10 κατηγορίες {'T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot'}.

Αρχικά, εκτελέσαμε τον κώδικα του αρχείου “fashion.py” με epochs 400 και αλγόριθμο βελτιστοποίησης “adam”.

Train Data

Model Definition

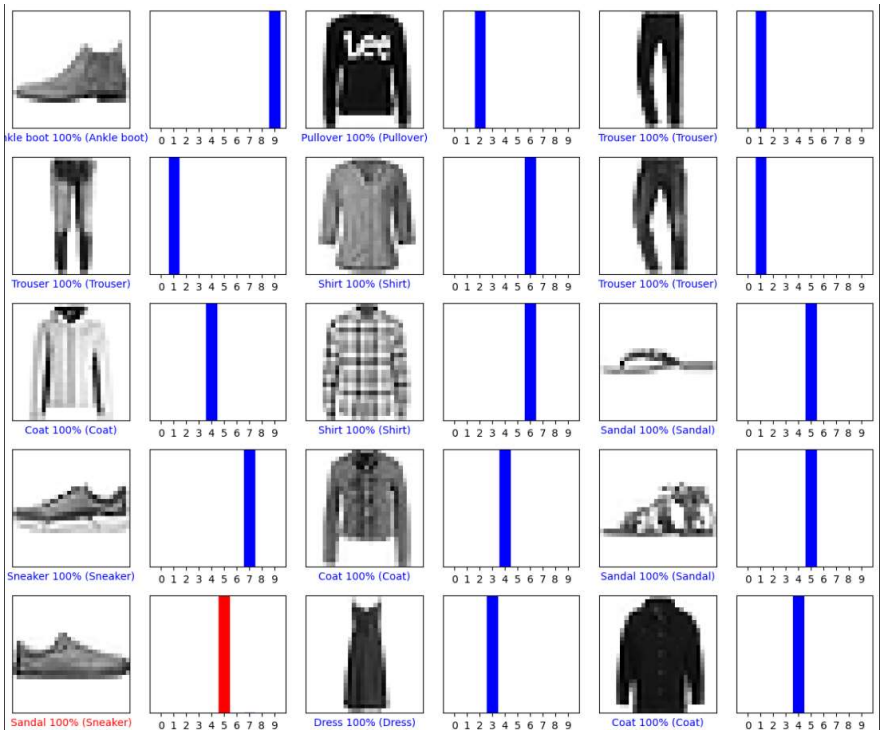
keras.layers.Flatten(input_shape=(28, 28)): This layer flattens the input images from 2D (28x28 pixels) to 1D (784 pixels) which is necessary for the dense layers.

keras.layers.Dense(128, activation='relu'): This is a dense layer with 128 neurons and ReLU activation function. It serves as a hidden layer.

keras.layers.Dense(10): This is the output layer with 10 neurons, corresponding to the 10 classes in the Fashion MNIST dataset. No activation function is specified so linear is used by default.

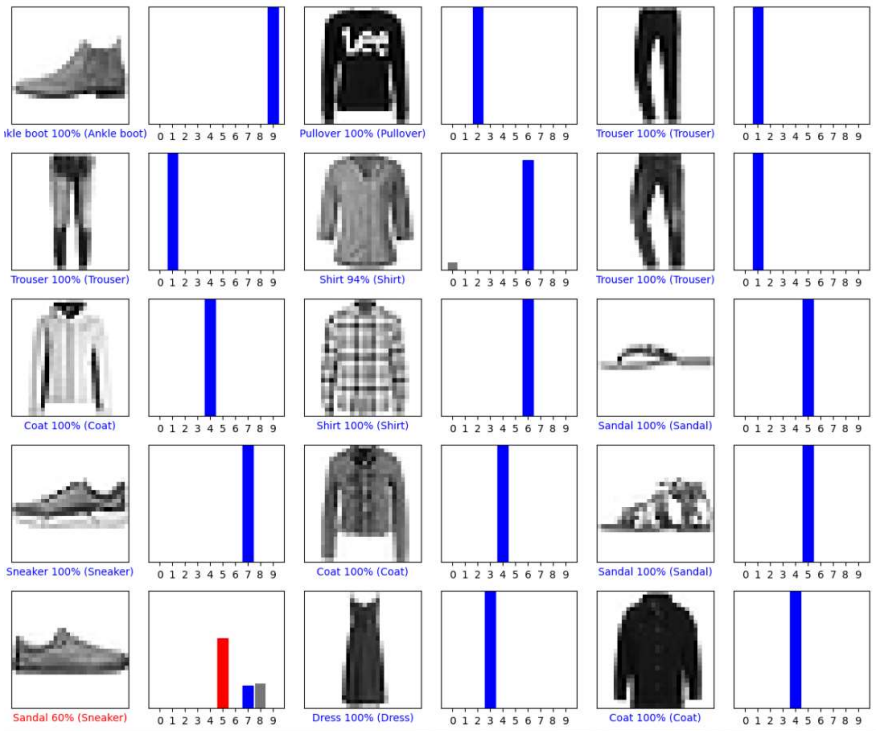
Δοκιμάσαμε τους αλγορίθμους βελτιστοποίησης adam, sgd, rmsprop, nadam, adamax, και ftrl. Παρακάτω φαίνονται τα αποτελέσματα για κάθε αλγόριθμο:

1. adam



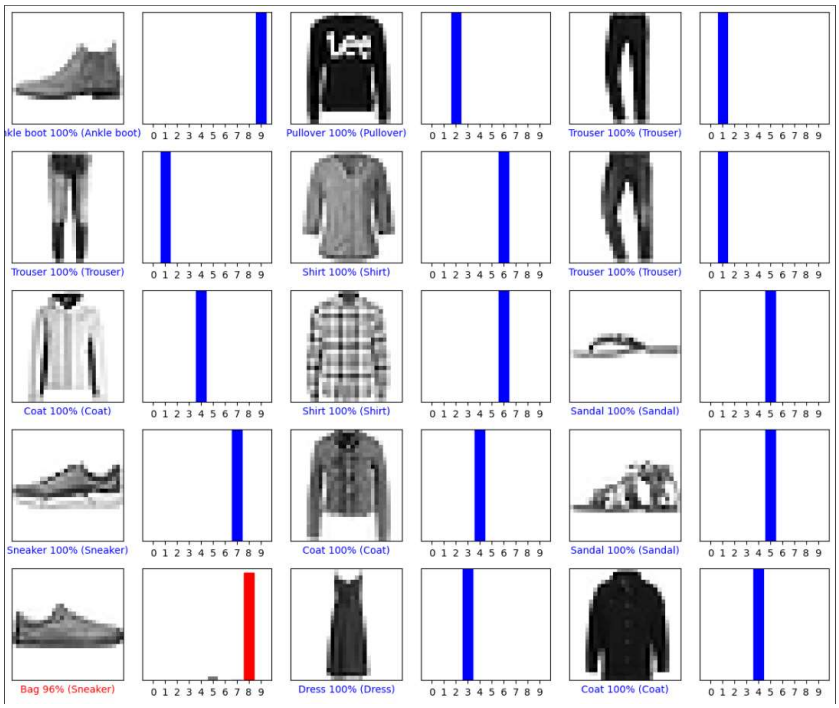

```
313/313 - 1s - loss: 1.8853 - accuracy: 0.8829 - 772ms/epoch - 2ms/step

Test accuracy: 0.8828999996185303
313/313 [==============================] - 1s 2ms/step
```

2. sgd



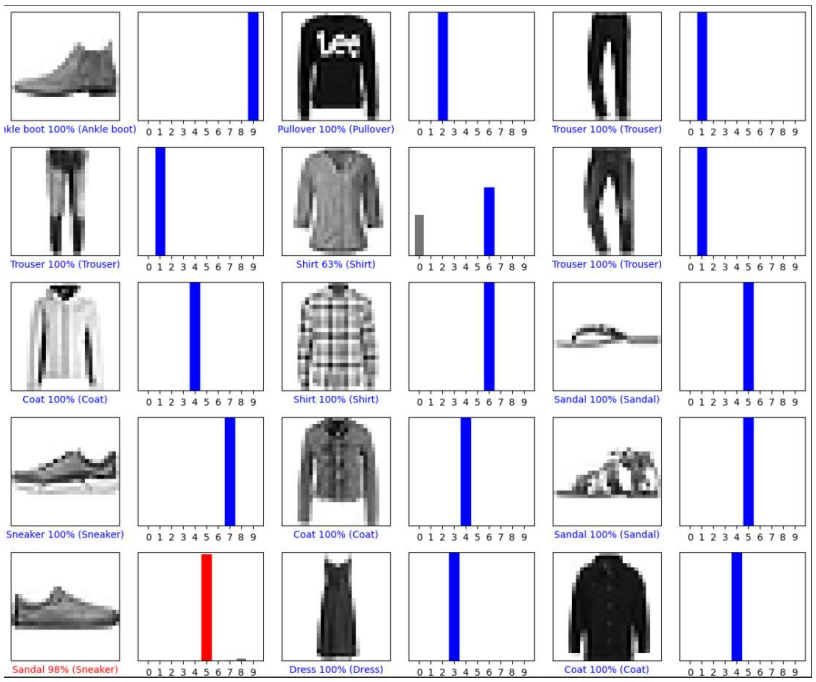

```
Test accuracy: 0.881600022315979
313/313 [==============================] - 1s 3ms/step
```

3. rmsprop



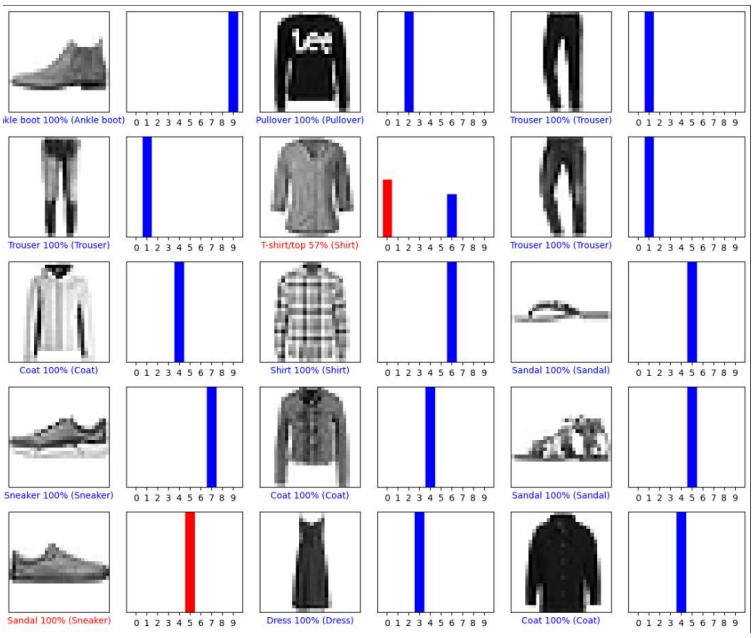

```
313/313 - 1s - loss: 2.9463 - accuracy: 0.8767 - 592ms/epoch - 2ms/step

Test accuracy: 0.8766999840736389
313/313 [==============================] - 1s 3ms/step
```

4. nadam



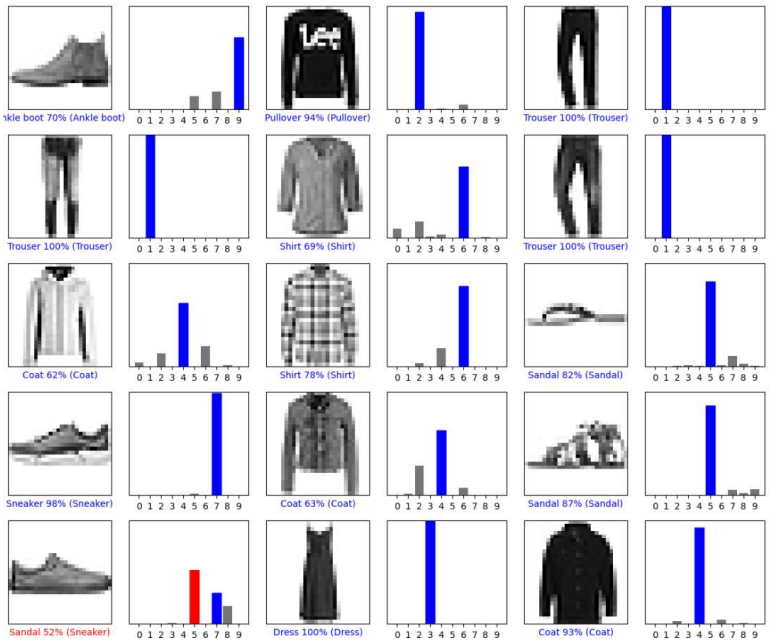

```
313/313 - 0s - loss: 1.8751 - accuracy: 0.8836 - 450ms/epoch - 1ms/step

Test accuracy: 0.8835999965667725
313/313 [==============================] - 1s 2ms/step
```

5. adamax




```
313/313 - 0s - loss: 0.9985 - accuracy: 0.8880 - 449ms/epoch - 1ms/step

Test accuracy: 0.8880000114440918
313/313 [==============================] - 1s 2ms/step
```

6. ftrl

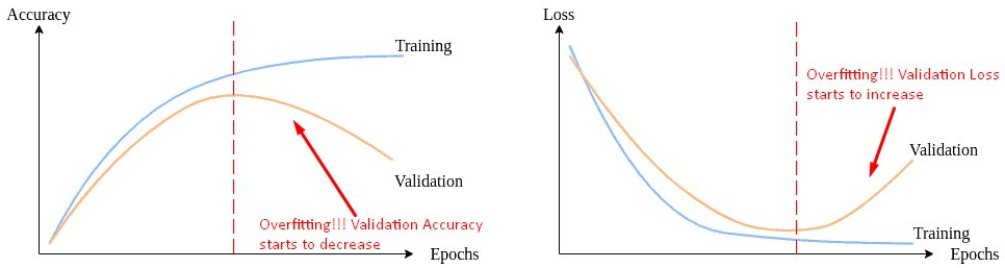



```
313/313 - 1s - loss: 0.4400 - accuracy: 0.8457 - 573ms/epoch - 2ms/step

Test accuracy: 0.8457000255584717
313/313 [==============================] - 1s 2ms/step
```

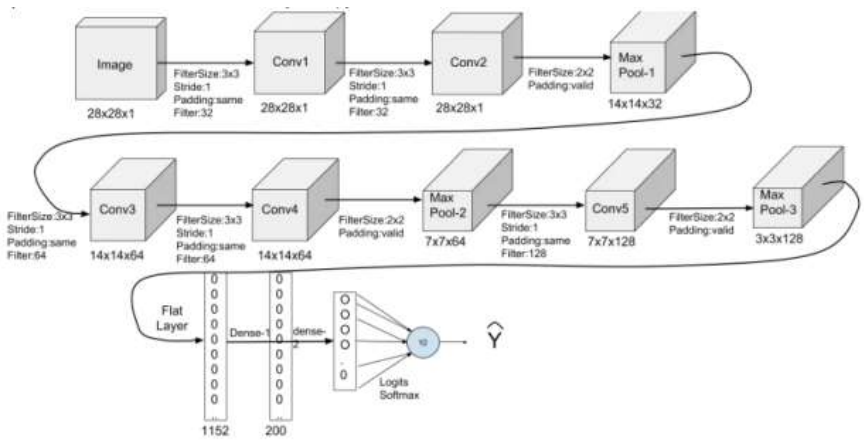
Με βάση τα παραπάνω αποτελέσματα, παρατηρούμε ότι το μεγαλύτερο accuracy επίσης το δίνει ο αλγόριθμος “adamax”(0.8880).

Η ακρίβεια στα δεδομένα εκπαίδευσης αυξάνεται σταθερά καθώς προχωρούν οι εποχές. Η ακρίβεια στα δεδομένα επικύρωσης επίσης αυξάνεται αρχικά, αλλά στη συνέχεια παρουσιάζει διακυμάνσεις και δεν βελτιώνεται μετά από ένα σημείο. Αυτό υποδηλώνει πιθανή υπερεκπαίδευση (overfitting), όπου το μοντέλο αποδίδει πολύ καλά στα δεδομένα εκπαίδευσης, αλλά δεν γενικεύει εξίσου καλά στα δεδομένα επικύρωσης.

Γενικά γνωρίζουμε:

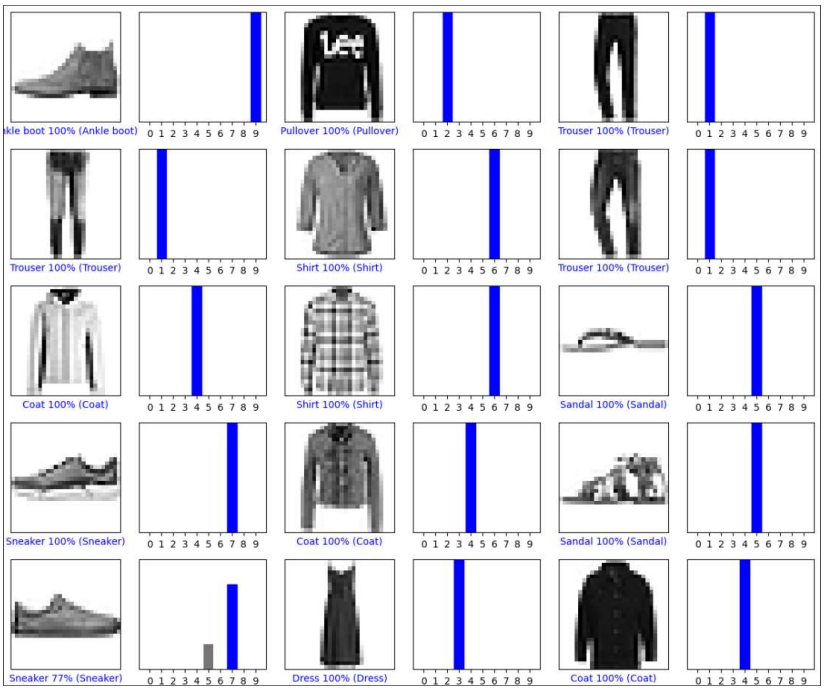


Έπειτα, αλλάξαμε την αρχιτεκτονική του δικτύου ώστε να υλοποιεί το παρακάτω σχήμα. Επίσης, αλλάξαμε το epoch από 400 σε 50 για να τελειώνει πιο γρήγορα η εκπαίδευση και χρησιμοποιήσαμε τον adam optimizer.




```
model = keras.Sequential([
    keras.layers.Conv2D(32, (3, 3), strides = 1, activation = 'relu', input_shape=(28, 28, 1), padding = 'same'),
    keras.layers.Conv2D(32, (3, 3), strides = 1, activation = 'relu', padding = 'same'),
    keras.layers.MaxPooling2D((2, 2), padding = 'valid'),

    keras.layers.Conv2D(64, (3, 3), strides = 1, activation = 'relu', padding ='same'),
    keras.layers.Conv2D(64, (3, 3), strides = 1, activation = 'relu', padding ='same'),
    keras.layers.MaxPooling2D((2, 2), padding = 'valid'),

    keras.layers.Conv2D(128, (3, 3), strides = 1, activation = 'relu', padding ='same'),
    keras.layers.Conv2D(128, (3, 3), strides = 1, activation = 'relu', padding ='same'),
    keras.layers.MaxPooling2D((2, 2), padding = 'valid'),

    keras.layers.Flatten(),
    keras.layers.Dense(200, activation ='relu'),
    keras.layers.Dense(10)
])
```

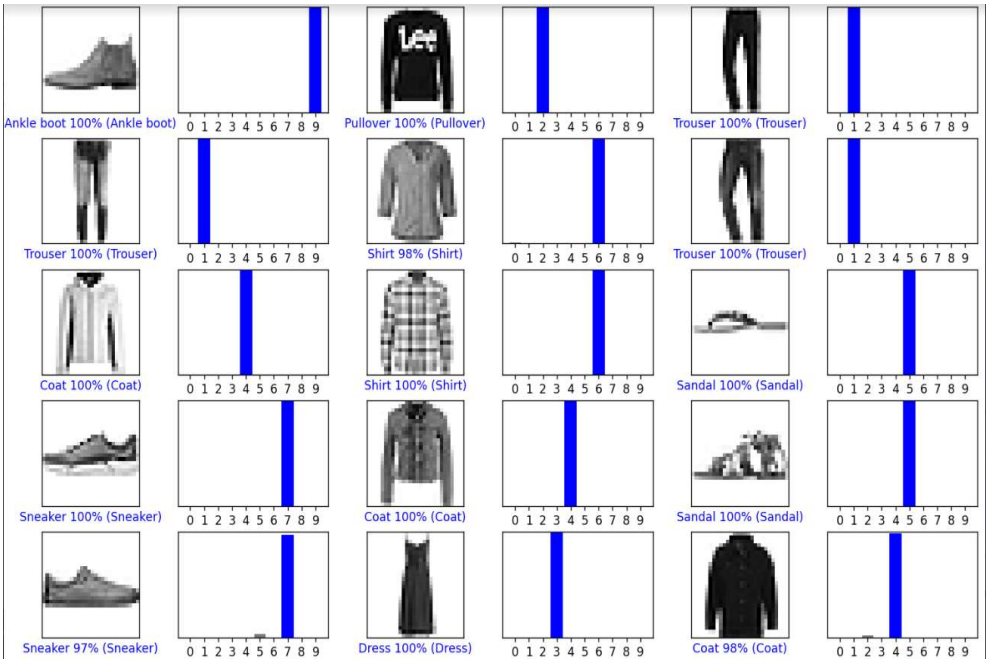
Αρχικά έχουμε (Χωρίς προσθήκες BatchNormalization και DropOuts):




```
313/313 - 13s - loss: 0.7099 - accuracy: 0.9220 - 13s/epoch - 42ms/step

Test accuracy: 0.921999990940094
313/313 [==============================] - 14s 45ms/step
```

Έπειτα, αλλάξαμε το μοντέλο προσθέτοντας BatchNormalization μετά τα επίπεδα conv1, conv2, conv3, conv4, conv5, dense1:

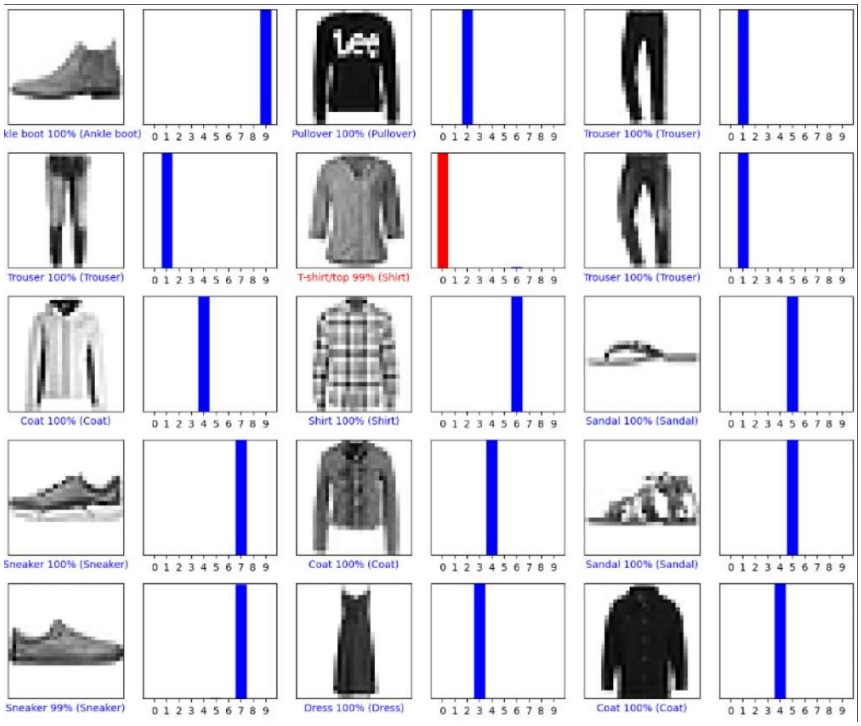
```
model = keras.Sequential([
    keras.layers.Conv2D(32, (3, 3), strides = 1, activation = 'relu', input_shape=(28, 28, 1), padding = 'same'),
    keras.layers.BatchNormalization(),
    keras.layers.Conv2D(32, (3, 3), strides = 1, activation = 'relu', padding = 'same'),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D((2, 2), padding = 'valid'),

    keras.layers.Conv2D(64, (3, 3), strides = 1, activation = 'relu', padding ='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Conv2D(64, (3, 3), strides = 1, activation = 'relu', padding ='same'),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D((2, 2), padding = 'valid'),

    keras.layers.Conv2D(128, (3, 3), strides = 1, activation = 'relu', padding ='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Conv2D(128, (3, 3), strides = 1, activation = 'relu', padding ='same'),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D((2, 2), padding = 'valid'),
    keras.layers.Flatten(),
    keras.layers.Dense(200, activation ='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(10)
])
```




```
313/313 - 3s - 9ms/step - accuracy: 0.9311 - loss: 0.4583

Test accuracy: 0.9311000108718872
```

Δοκιμάσαμε να τοποθετήσουμε το Batch Normalization πριν την συνάρτηση ReLU (Δηλαδή Conv χωρίς ReLU -> BatchNormalization -> ReLU).

```
model = keras.Sequential([
    keras.layers.Conv2D(32, (3, 3), strides=1, activation=None, input_shape=(28, 28, 1), padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),

    keras.layers.Conv2D(32, (3, 3), strides=1, activation=None, padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),
    keras.layers.MaxPooling2D((2, 2), padding='valid'),

    keras.layers.Conv2D(64, (3, 3), strides=1, activation=None, padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),

    keras.layers.Conv2D(64, (3, 3), strides=1, activation=None, padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),
    keras.layers.MaxPooling2D((2, 2), padding='valid'),

    keras.layers.Conv2D(128, (3, 3), strides=1, activation=None, padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),

    keras.layers.Conv2D(128, (3, 3), strides=1, activation=None, padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),
    keras.layers.MaxPooling2D((2, 2), padding='valid'),

    keras.layers.Flatten(),
    keras.layers.Dense(200,activation = 'relu'),
    keras.layers.Dense(10)
])
```

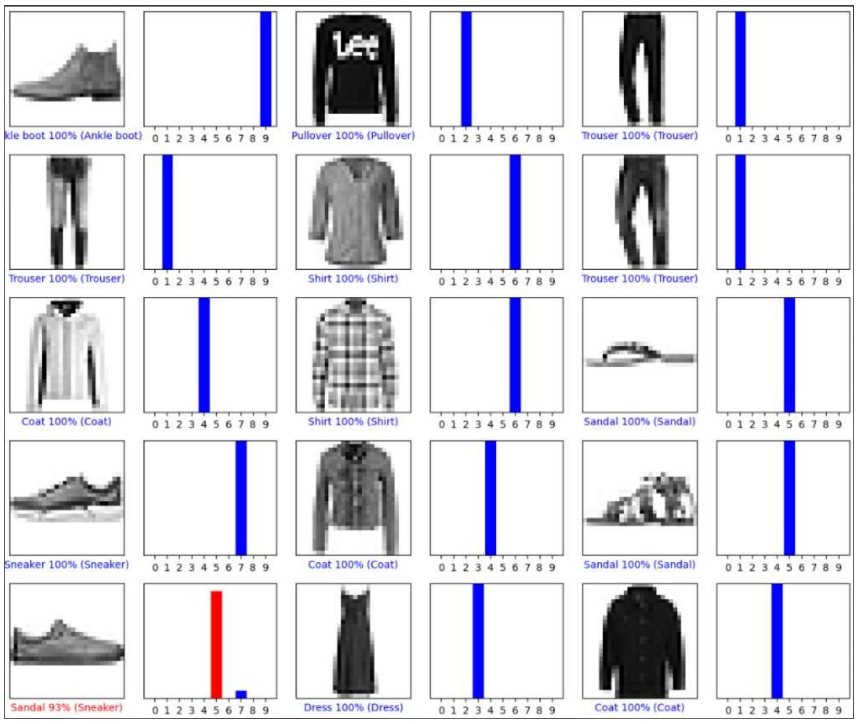



```
313/313 - 14s - loss: 0.4845 - accuracy: 0.9344 - 14s/epoch - 44ms/step

Test accuracy: 0.9344000220298767
313/313 [==============================] - 18s 52ms/step
```

Παρατηρήσαμε ότι αν τοποθετήσουμε το Batch Normalization πριν την συνάρτηση ReLU τότε η ακρίβεια αυξάνεται οπότε προχωρήσαμε κρατώντας το μοντέλο μας έτσι.

Τέλος, προσθέτουμε dropout(0.2) αμέσως μετά το πρώτο maxPooling, dropout(0.3) αμέσως μετά το δεύτερο maxPooling, dropout(0.4) αμέσως μετά το τρίτο maxPooling και dropout(0.5) αμέσως πριν το τελευταίο dense(10) layer.

```
model = keras.Sequential([
    keras.layers.Conv2D(32, (3, 3), strides=1, activation=None, input_shape=(28, 28, 1), padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),

    keras.layers.Conv2D(32, (3, 3), strides=1, activation=None, padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),
    keras.layers.MaxPooling2D((2, 2), padding='valid'),
    keras.layers.Dropout(0.2),

    keras.layers.Conv2D(64, (3, 3), strides=1, activation=None, padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),

    keras.layers.Conv2D(64, (3, 3), strides=1, activation=None, padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),
    keras.layers.MaxPooling2D((2, 2), padding='valid'),
    keras.layers.Dropout(0.3),

    keras.layers.Conv2D(128, (3, 3), strides=1, activation=None, padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),

    keras.layers.Conv2D(128, (3, 3), strides=1, activation=None, padding='same'),
    keras.layers.BatchNormalization(),
    keras.layers.Activation('relu'),
    keras.layers.MaxPooling2D((2, 2), padding='valid'),
    keras.layers.Dropout(0.4),

    keras.layers.Flatten(),
    keras.layers.Dense(200,activation = 'relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(10)
])
```




```
Epoch 50/50
1875/1875 [==============================] - 418s 223ms/step - loss: 0.0958 - accuracy: 0.9647 - val_loss: 0.1988 - val_accuracy: 0.9428
313/313 - 16s - loss: 0.1988 - accuracy: 0.9428 - 16s/epoch - 50ms/step

Test accuracy: 0.942799985408783
313/313 [==============================] - 19s 59ms/step
```

Με βάση τα παραπάνω αποτελέσματα, μπορούμε να παρατηρήσουμε ότι με την προσθήκη των Branch Normalization και των Dropouts, η ακρίβεια του δικτύου γίνεται καλύτερη έχοντας εν τέλη ακρίβεια περίπου ίση με 94.28%.